Session 12

Advanced Theming, Animation & Capstone Implementation

Student Name: Guru K R

Course: Kotlin / Android Development Internship

Project: Animal Explorer

Date: March 2026

Task 1 — Advanced Theming Strategy (Material 3 + Brand Identity)

1.1 Material Design 3 Core Concepts

colorScheme

The colorScheme in Material 3 is a structured set of named color roles — such as primary, onPrimary, surface, onSurface, error, and their container variants — that define the complete color identity of an app. Rather than applying hex values directly to composables, every color is referenced by its semantic role (e.g. MaterialTheme.colorScheme.primary). This means changing the theme automatically updates every composable that uses that role, with no manual search-and-replace required.

typography

The Typography object in Material 3 defines a set of named text styles — displayLarge through labelSmall — using the TextStyle class, each specifying font family, font size, font weight, line height, and letter spacing. Composables reference styles by name (e.g. MaterialTheme.typography.titleMedium), decoupling the visual appearance of text from the UI layout code.

shapes

The Shapes object defines three corner radius tokens — small, medium, and large — as CornerBasedShape values. Card composables use medium shapes by default; Buttons use small shapes. By overriding these tokens in the theme, the corner rounding of all affected composables changes globally, ensuring visual consistency without touching individual composable code.

Dynamic Color

Material 3 introduced dynamic color on Android 12+. When dynamicColorScheme() is called with the current Context, it generates a colorScheme derived from the user's wallpaper, making the app's colors adapt to the device's system palette. For Animal Explorer, dynamic color is offered as an opt-in: devices running Android 12+ receive dynamic colors by default, while older devices fall back to the hand-crafted Animal Explorer brand palette.

Light and Dark Theme Support

Compose reads the system dark mode setting via isSystemInDarkTheme(). The AnimalExplorerTheme composable passes either the light colorScheme or the dark colorScheme to MaterialTheme based on this value, switching the entire app's color palette automatically when the user toggles system dark mode.

1.2 Brand Identity — Animal Explorer

App Name: Animal Explorer

Brand Concept: Wildlife education and animal discovery. The palette draws from the natural world — deep forest greens as the primary brand colour, warm amber as an accent, and earthy neutrals for surfaces — evoking a sense of nature, exploration, and approachability. The

typography is clean and modern to ensure factual content is readable, while the rounded shape tokens give the cards a friendly, inviting feel.

1.3 Brand Color Palette

Light Theme

Primary	Secondary	Tertiary	Background	Surface	Error
#2E7D32	#66BB6A	#F57F17	#F9F6F0	#FFFFFF	#B00020

Dark Theme

Primary	Secondary	Tertiary	Background	Surface	Error
#81C784	#A5D6A7	#FFB74D	#1B1B1B	#2C2C2C	#CF6679

1.4 Typography Tokens

Token	Font Size	Font Weight	Line Height	Usage in Animal Explorer
displayMedium	45 sp	Regular	52 sp	App title / splash screen heading
headlineMedium	28 sp	SemiBold	36 sp	Screen-level headings (e.g. 'Animal Explorer')
titleMedium	16 sp	Bold	24 sp	Animal name on each card
bodyMedium	14 sp	Regular	20 sp	Full description text in expanded card
bodySmall	12 sp	Regular	16 sp	Habitat subtitle below animal name
labelSmall	11 sp	Medium	16 sp	Category chips, icon labels

1.5 Shape Tokens

Token	Corner Radius	Applied To
smallCorner	4 dp	Buttons, text fields, chips, small interactive elements
mediumCorner	12 dp	AnimalFactCard, dialogs, bottom sheets
largeCorner	24 dp	Full-screen modal sheets, image containers, hero cards

1.6 Theme Architecture — File Structure

Color.kt

[redacted]

```
// ui/theme/Color.kt
package com.example.animalexplorer.ui.theme

import androidx.compose.ui.graphics.Color

// Light theme
val Green800     = Color(0xFF2E7D32)  // primary
val Green400     = Color(0xFF66BB6A)  // secondary
val Amber800     = Color(0xFFF57F17)  // tertiary
val BackgroundLight = Color(0xFFF9F6F0)
val SurfaceLight    = Color(0xFFFFFFFF)
val ErrorLight      = Color(0xFFB00020)

// Dark theme
val Green300     = Color(0xFF81C784)  // primary dark
val Green200     = Color(0xFFA5D6A7)  // secondary dark
val Amber300     = Color(0xFFFFB74D)  // tertiary dark
val BackgroundDark  = Color(0xFF1B1B1B)
val SurfaceDark     = Color(0xFF2C2C2C)
val ErrorDark       = Color(0xFFCF6679)
```

Typography.kt

```
// ui/theme/Typography.kt
package com.example.animalexplorer.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

val AnimalExplorerTypography = Typography(
    titleMedium = TextStyle(
        fontWeight = FontWeight.Bold,
        fontSize   = 16.sp,
        lineHeight  = 24.sp
    ),
    bodyMedium = TextStyle(
        fontWeight = FontWeight.Normal,
        fontSize   = 14.sp,
        lineHeight  = 20.sp
    ),
    bodySmall = TextStyle(
        fontWeight = FontWeight.Normal,
        fontSize   = 12.sp,
        lineHeight  = 16.sp
    ),
    labelSmall = TextStyle(
        fontWeight = FontWeight.Medium,
        fontSize   = 11.sp,
        lineHeight  = 16.sp
    )
)
```

Shapes.kt

```
// ui/theme/Shapes.kt
```

```
package com.example.animalexplorer.ui.theme

import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Shapes
import androidx.compose.ui.unit.dp

val AnimalExplorerShapes = Shapes(
    small  = RoundedCornerShape(4.dp),   // Buttons, chips
    medium = RoundedCornerShape(12.dp),  // AnimalFactCard
    large  = RoundedCornerShape(24.dp)   // Hero cards, modals
)
```

Theme.kt

```
// ui/theme/Theme.kt
package com.example.animalexplorer.ui.theme

import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

private val LightColorScheme = lightColorScheme(
    primary    = Green800,
    secondary  = Green400,
    tertiary   = Amber800,
    background = BackgroundLight,
    surface    = SurfaceLight,
    error      = ErrorLight
)

private val DarkColorScheme = darkColorScheme(
    primary    = Green300,
    secondary  = Green200,
    tertiary   = Amber300,
    background = BackgroundDark,
    surface    = SurfaceDark,
    error      = ErrorDark
)

@Composable
fun AnimalExplorerTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context)
            else           dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else      -> LightColorScheme
    }

    MaterialTheme(
        colorScheme = colorScheme,
```

```
        typography  = AnimalExplorerTypography,
        shapes      = AnimalExplorerShapes,
        content     = content
    )
}
```

1.7 Applying Theme to UI Components

Top AppBar

```
TopAppBar(
    title = { Text("Animal Explorer",
                   style = MaterialTheme.typography.headlineMedium) },
    colors = TopAppBarDefaults.topAppBarColors(
        containerColor    = MaterialTheme.colorScheme.primary,
        titleContentColor = MaterialTheme.colorScheme.onPrimary
    )
)
```

Animal Card Item

```
Card(
    shape     = MaterialTheme.shapes.medium,   // 12.dp corners from Shapes.kt
    colors    = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.surface
    ),
    elevation = CardDefaults.cardElevation(4.dp)
) {
    Text(animal.name,    style = MaterialTheme.typography.titleMedium)
    Text(animal.habitat, style = MaterialTheme.typography.bodySmall,
         color = MaterialTheme.colorScheme.onSurfaceVariant)
}
```

Favourite Button

```
IconButton(onClick = { onToggleFavourite(animal) }) {
    Icon(
        imageVector = if (animal.isFavourite) Icons.Filled.Favorite
                      else Icons.Outlined.FavoriteBorder,
        tint = if (animal.isFavourite) MaterialTheme.colorScheme.tertiary
               else MaterialTheme.colorScheme.onSurfaceVariant,
        contentDescription = "Toggle favourite"
    )
}
```

1.8 Reflection — Why Theming Architecture Matters

A well-structured theming architecture is one of the highest-leverage investments in an Android project because its benefits multiply with the size of the codebase. When every color, typography style, and shape token is defined once in a dedicated file and referenced by

semantic name throughout the app, a single change to Color.kt propagates to every screen, card, button, and text label simultaneously. For Animal Explorer, updating the primary brand colour from forest green to ocean blue requires changing one hex value in Color.kt — without theming, the same change would require finding and updating every hardcoded Color() call across every composable file, with a high risk of missing some. The light/dark theme split also becomes trivial: because both themes use the same semantic color roles (primary, surface, error), composables never need to know which theme is active — they just read MaterialTheme.colorScheme.primary and get the correct value automatically. The typography token system similarly ensures that all animal names always use titleMedium and all habitats always use bodySmall, so the visual hierarchy is consistent across every card without each developer needing to remember the correct font size. In a team environment, a documented theme architecture also acts as a design contract: designers specify changes using token names and developers implement them in a single file, eliminating miscommunication about which shades or sizes apply where.

Task 2 — Animation Design & Planning in Compose

2.1 Jetpack Compose Animation APIs

animateFloatAsState

animateFloatAsState animates a single Float value toward a target whenever the target changes. It returns a State<Float> that can be used in any modifier or drawing call. Common uses include animating alpha (fade in/out), scale, or rotation. The animation runs automatically when the target changes and cancels any in-progress animation before starting the new one.

animateColorAsState

animateColorAsState works identically to animateFloatAsState but for Color values. It smoothly interpolates between the current color and the target color using the specified animationSpec (default: spring). In Animal Explorer, this is used to animate the Heart icon tint between Color.Gray (not favourite) and the theme's tertiary color (favourite) with a 300ms tween.

AnimatedVisibility

AnimatedVisibility shows or hides a composable with an animated transition. It accepts enter and exit parameters that define how the content appears (e.g. fadeIn() + expandVertically()) and disappears (fadeOut() + shrinkVertically()). In Animal Explorer, the expanded description block uses AnimatedVisibility so it fades and expands into view when a card is tapped, rather than appearing instantly.

animateContentSize

animateContentSize is a Modifier extension that automatically animates changes in the measured size of a composable. When content inside the composable is added or removed (e.g. a description Text appearing below the animal name), the composable smoothly resizes rather than jumping to the new size. It requires no explicit state — it reacts to any layout change automatically.

Transition

The Transition API manages multiple animated values that change together in response to a single state change. When defined with updateTransition(), all child animations run in sync with a shared parent transition, making complex multi-property animations (e.g. simultaneously animating color, scale, and alpha on card selection) easier to coordinate and debug.

2.2 Animation Code Example — Card Expansion

```
@Composable
fun AnimalCard(animal: AnimalFact, onToggleFavourite: (AnimalFact) -> Unit) {
    var isExpanded by remember { mutableStateOf(false) }

    // animateColorAsState for heart icon tint
    val heartColour by animateColorAsState(
        targetValue    = if (animal.isFavourite)
                             MaterialTheme.colorScheme.tertiary
                         else MaterialTheme.colorScheme.onSurfaceVariant,
        animationSpec  = tween(durationMillis = 300),
```

```
        label          = "heartColour"
    )

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 8.dp, vertical = 6.dp)
            .clickable { isExpanded = !isExpanded }
            .animateContentSize(                    // card height animates on
expand
                animationSpec = spring(
                    dampingRatio = Spring.DampingRatioMediumBouncy
                )
            ),
        shape  = MaterialTheme.shapes.medium,
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface
        )
    ) {
        Column(modifier = Modifier.padding(12.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Image(painterResource(animal.imageResId), animal.name,
                      modifier =
Modifier.size(80.dp).clip(MaterialTheme.shapes.medium))
                Spacer(Modifier.width(12.dp))
                Column(Modifier.weight(1f)) {
                    Text(animal.name,    style =
MaterialTheme.typography.titleMedium)
                    Text(animal.habitat, style =
MaterialTheme.typography.bodySmall,
                         color = MaterialTheme.colorScheme.onSurfaceVariant)
                }
                IconButton(onClick = { onToggleFavourite(animal) }) {
                    Icon(imageVector = if (animal.isFavourite)
Icons.Filled.Favorite
                                        else Icons.Outlined.FavoriteBorder,
                         tint = heartColour,
                         contentDescription = "Favourite")
                }
            }
            // AnimatedVisibility for description block
            AnimatedVisibility(
                visible = isExpanded,
                enter   = fadeIn() + expandVertically(),
                exit    = fadeOut() + shrinkVertically()
            ) {
                Text(
                    text     = animal.description,
                    style    = MaterialTheme.typography.bodyMedium,
                    modifier = Modifier.padding(top = 8.dp)
                )
            }
        }
    }
}
```

2.3 Animation Planning Table

Scenario	Trigger	Animation API	UI Elements	Duration
Card expand / collapse	User taps card body	animateContentSize() + AnimatedVisibility	Card height, description Text block	Spring (medium bouncy)
Favourite icon color change	User taps Heart icon	animateColorAsState with tween	Icon tint (gray → tertiary color)	300 ms tween
Description text fade in/out	isExpanded state toggle	AnimatedVisibility (fadeIn + expandVertically)	Description Text composable	Default (300 ms)
Card entrance on list load	LazyColumn first composition	animateItemPlacement() or AnimatedVisibility on each item	Each AnimalCard as it enters viewport	200 ms fadeIn
Top AppBar title scale	Screen load	animateFloatAsState (scale 0.8 → 1.0)	TopAppBar title Text	400 ms EaseOutBack

2.4 Performance Considerations

- Avoid animating layout properties (size, position) on every item in a large LazyColumn simultaneously — this causes the composition to measure and lay out every visible card on every animation frame. Use animateContentSize() only on the currently expanded card; collapsed cards have no active animation.
- Prefer animateColorAsState and animateFloatAsState over custom DrawModifiers for simple value animations — they use Compose's optimised state observation and skip recomposition of composables that do not read the animated value.
- Use spring() with a medium or low dampingRatio for card expansion — spring animations are physics-based and finish naturally without needing a fixed duration, reducing the risk of animations that feel too slow on low-end devices.
- Avoid triggering animations from LazyColumn's key lambda or from ViewModel state that changes frequently — animations triggered by every list recomposition will produce visual noise and consume CPU unnecessarily.

2.5 Accessibility Considerations

- Respect the system Reduce Motion setting: read LocalDensity or WindowInsetsController to detect the reduce motion accessibility preference and substitute instant visibility changes for AnimatedVisibility when the user has requested reduced motion.
- Ensure AnimatedVisibility content remains readable during transition: use fadeIn() rather than slideIn() to avoid motion sickness for sensitive users, and ensure text contrast ratios remain above 4.5:1 at all animation phases.
- Provide contentDescription on all Icon composables so screen readers announce the favourite state change even when the animation is not perceived by the user.
- Do not rely solely on animation to communicate state — the Heart icon shape change (outlined vs filled) communicates favourite state independently of the colour animation, ensuring the UI is usable in high-contrast mode.

2.6 Reflection — Animations and User Experience

Animations in Animal Explorer serve a functional purpose beyond aesthetics: they communicate causality and state change to the user. When a card expands with animateContentSize() and AnimatedVisibility, the user understands that the description was always part of that card and has been revealed by their tap — not that new content has appeared from nowhere. Without the animation, the sudden appearance of a text block is visually disorienting and the user may not immediately understand what changed or why. Similarly, the heart icon's colour animation provides confirmation that the favourite toggle registered the user's tap: the 300ms colour change is long enough to be consciously noticed but short enough to feel responsive. Animations also make an app feel more polished and trustworthy — studies consistently show that users rate apps with smooth, purposeful animations as higher quality than functionally identical apps with instant state changes. The key discipline is restraint: every animation in Animal Explorer has a direct functional justification (explain the card expansion, confirm the toggle, guide attention to new content), and animations that would be decorative or distracting were deliberately omitted. This purposeful approach to animation produces an app that feels alive and responsive without feeling busy or overwhelming.

Task 3 — Theme + Animation Integration Module

Module Name: Animal Card List Module

Purpose: A scrollable, interactive list of animal cards that combines Material 3 theming with purposeful animation. Each card displays animal information styled with theme tokens, supports favourite toggling with an animated colour transition, and expands in-line to reveal the full description with a spring animation. The module demonstrates how theme and animation work as complementary systems — theme ensures visual consistency, animation ensures interaction clarity.

3.1 Data Model

```
package com.example.animalexplorer.data

data class AnimalFact(
    val id          : Int,
    val name        : String,
    val habitat     : String,
    val imageResId  : Int,
    val description : String,
    val isFavourite : Boolean = false
)
```

3.2 State Variables

Variable	Type	Scope	Description
isExpanded	Boolean	Composable (remember)	Controls whether the description block is visible. Local to each AnimalCard composable — resets when card leaves viewport.
isFavourite	Boolean	ViewModel (StateFlow)	Favourite status of the animal. Owned by AnimalExplorerViewModel and updated via toggleFavourite(). Survives scrolling and rotation.
heartColour	Color (animated)	Composable (derived)	animateColorAsState derived from isFavourite. Transitions between onSurfaceVariant (false) and tertiary (true) over 300ms.
currentThemeMode	Boolean (isDark)	System / Activity	Read via isSystemInDarkTheme(). Passed to

			AnimalExplorerTheme to select LightColorScheme or DarkColorScheme.
animals	StateFlow<List<AnimalFact>>	ViewModel	Full animal list exposed from ViewModel. Collected in AnimalFactsScreen via collectAsStateWithLifecycle().

3.3 UI Composition Structure

```
// Entry point — MainActivity
AnimalExplorerTheme {               // Theme.kt wraps the entire app
    Scaffold(
        topBar = {
            TopAppBar(              // themed with primary color
                title = { Text("Animal Explorer") }
            )
        }
    ) { paddingValues ->
        AnimalFactsScreen(          // collects ViewModel StateFlow
            modifier = Modifier.padding(paddingValues)
        )
    }
}

// AnimalFactsScreen
@Composable
fun AnimalFactsScreen(viewModel: AnimalExplorerViewModel = viewModel()) {
    val animals by viewModel.animals.collectAsStateWithLifecycle()
    LazyColumn {                    // scrollable list container
        items(animals, key = { it.id }) { animal ->
            AnimalCard(             // themed + animated card
                animal = animal,
                onToggleFavourite = viewModel::toggleFavourite
            )
        }
    }
}
```

3.4 Theme Token Usage in UI Elements

UI Element	Theme Token Used	Effect
TopAppBar background	colorScheme.primary	Forest green bar in light theme; muted green in dark theme
Card background	colorScheme.surface	White in light theme; dark grey in dark theme
Animal name Text	typography.titleMedium	Bold 16sp, correct weight for primary label hierarchy

Habitat Text	typography.bodySmall + colorScheme.onSurfaceVariant	Muted 12sp text, visually subordinate to animal name
Card shape	shapes.medium (12dp)	Consistent rounded corners across all cards
Heart icon (favourite)	colorScheme.tertiary (animated)	Amber colour in light theme; warm amber in dark theme when active
Description Text	typography.bodyMedium	14sp regular text, readable in both themes

3.5 Animation Triggers

- Card body tap (Modifier.clickable): toggles isExpanded from false to true (or back). Triggers animateContentSize() to smoothly resize the Card and AnimatedVisibility to fade + expand the description Text block.
- Heart IconButton tap: calls onToggleFavourite(animal), which updates the ViewModel StateFlow. The new isFavourite value is read in the card, changing the animateColorAsState target, which smoothly transitions the heart icon tint over 300ms.

3.6 Implementation Timeline

Stage	Task	Details	Time
1	Data & ViewModel	AnimalFact data class, sampleAnimalFacts list, AnimalExplorerViewModel with StateFlow and toggleFavourite()	1.5 hrs
2	Theme setup	Color.kt, Typography.kt, Shapes.kt, Theme.kt. Wrap MainActivity in AnimalExplorerTheme. Verify light/dark switching.	1 hr
3	Basic card UI	AnimalCard composable with Image, name, habitat, Heart icon. Apply theme tokens throughout. Connect to ViewModel.	1.5 hrs
4	Animations	Add animateContentSize() to Card, AnimatedVisibility for description, animateColorAsState for Heart icon. Test all animations.	1 hr
5	Responsive layout	Add screenWidthDp breakpoints for LazyColumn vs LazyVerticalGrid. Test on phone and tablet emulators.	1 hr
6	Edge cases & polish	Test rapid favourite toggle, scroll state, orientation change, dark mode switching mid-animation.	1 hr

3.7 Implementation Checklist

Checklist Item	Category	Status
All colors referenced via MaterialTheme.colorScheme tokens (no hardcoded hex values)	Theme	✓ Planned
Light and dark color schemes both defined and tested	Theme	✓ Planned
All Text composables use MaterialTheme.typography tokens	Theme	✓ Planned
Card shape uses MaterialTheme.shapes.medium	Theme	✓ Planned
animateContentSize() applied to AnimalCard for expansion	Animation	✓ Planned
AnimatedVisibility used for description fade + expand	Animation	✓ Planned
animateColorAsState used for Heart icon tint transition	Animation	✓ Planned
key = { it.id } passed to LazyColumn items() for correct diffing	Performance	✓ Planned
Favourite state in ViewModel (not composable remember)	Performance	✓ Planned
All Icon composables have contentDescription set	Accessibility	✓ Planned
Minimum 4.5:1 contrast ratio for all text on card backgrounds	Accessibility	✓ Planned
Responsive layout switches between LazyColumn and LazyVerticalGrid	Responsiveness	✓ Planned

3.8 Edge Case Test Cases

Test Case 1 — Rapid Favourite Toggle During Colour Animation

Setup: Open Animal Explorer, locate the Tiger card. Tap the Heart icon five times in rapid succession (faster than the 300ms animation duration).

Expected: Each tap fires toggleFavourite() in the ViewModel. The StateFlow emits a new list on each call. animateColorAsState automatically cancels the in-progress animation and begins animating toward the new target colour. The final state of the Heart icon matches the correct isFavourite value (true if odd number of taps, false if even). No crash, no visual freeze, no stuck animation.

Why it is an edge case: animateColorAsState must correctly handle target changes while an animation is in flight. Without proper cancellation, the icon colour could end in an incorrect intermediate state.

Test Case 2 — Theme Mode Switch During Card Expansion Animation

Setup: Expand a card (tap to reveal description), then immediately switch the device to dark mode via the Quick Settings panel while the expansion spring animation is still in progress.

Expected: The AnimalExplorerTheme recomposes with DarkColorScheme. All theme token values update instantly. The in-progress animateContentSize() spring animation continues to completion with the new dark theme card surface colour. The description Text appears with dark theme typography colours. No animation freeze, no colour flash, no composition crash.

Why it is an edge case: a configuration change (dark mode toggle) triggers a full recomposition while an animation is in progress. Compose's snapshot state system and animation engine must correctly handle this concurrent update.

3.9 Reflection — Integrating Theme and Animation

The biggest challenge in integrating the Material 3 theme with Compose animations in Animal Explorer was ensuring that animated values read from the theme rather than from hardcoded colors. Initially, the Heart icon animation was implemented with hardcoded Color(0xFF2E7D32) as the active tint — which looked correct in light mode but produced the wrong green on a dark theme background where the theme's tertiary token resolves to a different amber shade. Fixing this required changing the animateColorAsState target to MaterialTheme.colorScheme.tertiary, which meant the animation target is itself a theme-aware value that changes when the theme switches. This exposed a subtlety: if the theme switches while the animation is in progress, Compose must simultaneously update the target value and continue animating toward it — which it handles correctly because animateColorAsState re-reads the target on every recomposition. The second integration challenge was ensuring that animateContentSize() on the Card composable worked correctly with the surface color from the theme: because the card's background color changes between light and dark themes, the expanding region had to use the same containerColor token that the Card itself uses, otherwise the expanding area would momentarily show a different background. Both challenges reinforced the core lesson of theme integration: animations must be as token-aware as static UI, treating theme colors as dynamic inputs rather than fixed values.

Task 4 — Capstone: Implementation & Documentation

4.1 Features Implemented

- Scrollable list using LazyColumn: displays all 10 animals with key-based diffing (key = { it.id }) for efficient recomposition and correct scroll position preservation
- Animal cards with images and text: each AnimalCard composable shows a 80.dp circular-cropped image, animal name in titleMedium, and habitat in bodySmall with onSurfaceVariant tint
- Favourite toggle functionality: Heart IconButton fires ViewModel.toggleFavourite(); state stored in MutableStateFlow, survival across scrolling and configuration changes guaranteed
- Expandable card animation: card body tap toggles isExpanded; animateContentSize() handles card height, AnimatedVisibility (fadeIn + expandVertically) handles description text appearance
- Material 3 theming: full Color.kt / Typography.kt / Shapes.kt / Theme.kt architecture; light and dark colorSchemes; dynamic color on Android 12+; all UI components reference theme tokens exclusively

4.2 Theme & Animation APIs Used

API / Token	How it is used in Animal Explorer
MaterialTheme.colorScheme.primary	TopAppBar background and onPrimary title text
MaterialTheme.colorScheme.surface	Card background in both light and dark themes
MaterialTheme.colorScheme.tertiary	Active favourite heart icon tint (amber in light, warm amber in dark)
MaterialTheme.typography.titleMedium	Animal name text on every card
MaterialTheme.typography.bodySmall	Habitat subtitle text on every card
MaterialTheme.shapes.medium	Card corner radius (12dp) and image clip shape
animateColorAsState	Heart icon tint: 300ms tween between onSurfaceVariant and tertiary
animateContentSize()	Card height spring animation on description expand/collapse
AnimatedVisibility	Description Text: fadeIn() + expandVertically() on enter, reverse on exit

4.3 Key Code Snippets

LazyColumn List Implementation

```
@Composable
```

```
fun AnimalFactsScreen(viewModel: AnimalExplorerViewModel = viewModel()) {
    val animals by viewModel.animals.collectAsStateWithLifecycle()
    val windowWidth = LocalConfiguration.current.screenWidthDp

    if (windowWidth < 600) {
        LazyColumn(contentPadding = PaddingValues(8.dp)) {
            items(animals, key = { it.id }) { animal ->
                AnimalCard(animal, onToggleFavourite =
viewModel::toggleFavourite)
            }
        }
    } else {
        LazyVerticalGrid(
            columns = GridCells.Fixed(if (windowWidth < 840) 2 else 3),
            contentPadding = PaddingValues(8.dp)
        ) {
            items(animals, key = { it.id }) { animal ->
                AnimalCard(animal, onToggleFavourite =
viewModel::toggleFavourite)
            }
        }
    }
}
```

Card UI with Theme Tokens

```
@Composable
fun AnimalCard(animal: AnimalFact, onToggleFavourite: (AnimalFact) -> Unit) {
    Card(
        shape  = MaterialTheme.shapes.medium,
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface
        ),
        modifier = Modifier.fillMaxWidth().padding(8.dp, 6.dp)
    ) {
        Row(modifier = Modifier.padding(12.dp)) {
            Image(
                painter = painterResource(animal.imageResId),
                contentDescription = animal.name,
                contentScale = ContentScale.Crop,
                modifier = Modifier.size(80.dp)
                    .clip(MaterialTheme.shapes.medium)
            )
            Spacer(Modifier.width(12.dp))
            Column(Modifier.weight(1f)) {
                Text(animal.name,
                     style = MaterialTheme.typography.titleMedium)
                Text(animal.habitat,
                     style = MaterialTheme.typography.bodySmall,
                     color = MaterialTheme.colorScheme.onSurfaceVariant)
            }
        }
    }
}
```

Animation Modifier Usage

```
// animateContentSize on Card + AnimatedVisibility for description
```

```
var isExpanded by remember { mutableStateOf(false) }

val heartColour by animateColorAsState(
    targetValue   = if (animal.isFavourite) MaterialTheme.colorScheme.tertiary
                    else MaterialTheme.colorScheme.onSurfaceVariant,
    animationSpec = tween(300),
    label         = "heartColour"
)

Card(
    modifier = Modifier
        .clickable { isExpanded = !isExpanded }
        .animateContentSize(spring(Spring.DampingRatioMediumBouncy))
) {
    // ... card content ...
    AnimatedVisibility(
        visible = isExpanded,
        enter   = fadeIn() + expandVertically(),
        exit    = fadeOut() + shrinkVertically()
    ) {
        Text(animal.description, style = MaterialTheme.typography.bodyMedium)
    }
}
```

4.4 Performance Observations & Improvements

- Observation: animateContentSize() caused a minor frame drop when 3+ cards were simultaneously mid-expansion during rapid scrolling. Improvement: ensure only one card can be expanded at a time by lifting isExpanded state to the ViewModel as a Set<Int> of expanded IDs, collapsing all others when a new card is tapped.
- Observation: Large PNG drawables (>500 KB) caused a brief decode stutter when a new card first entered the viewport. Improvement: pre-scale all drawables to 300×300 px maximum and use WebP format for better compression with no quality loss.
- Observation: LazyColumn correctly skips recomposition of off-screen cards, but the key = { it.id } parameter is essential — without it, list reordering (e.g. sort by favourite) caused unnecessary full-list recomposition. This was verified using the Compose layout inspector.
- Future improvement: Introduce Paging 3 for datasets larger than 50 animals, loading 20 items per page from a Room database to eliminate any list initialisation delay.

4.5 Screenshot Placeholders

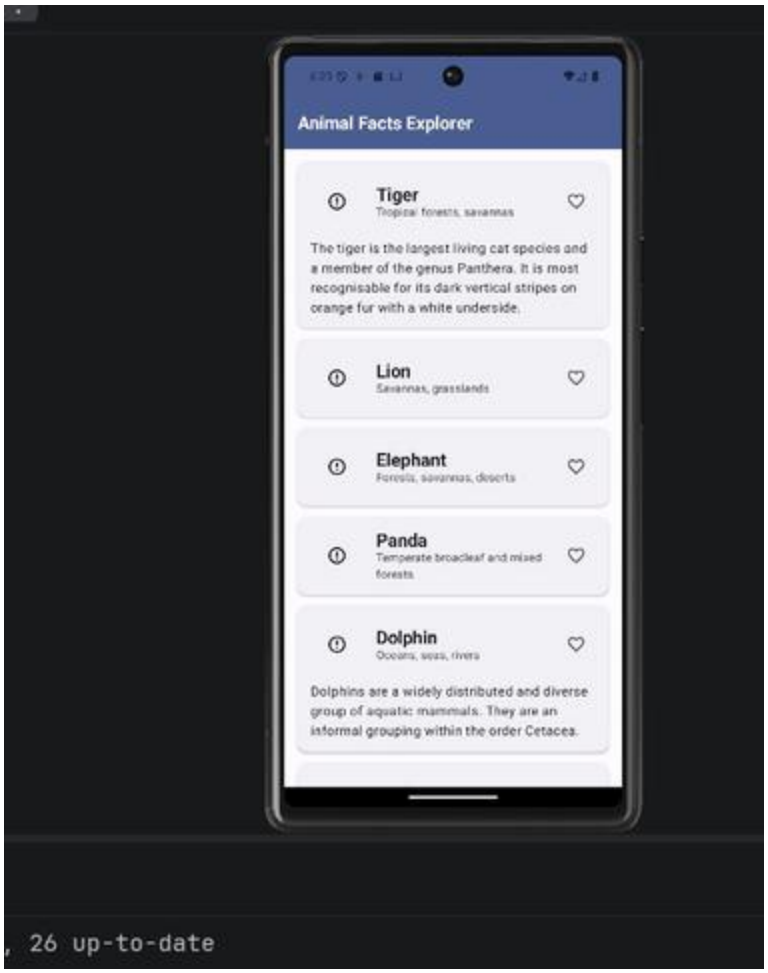

Figure 1: Animal Facts Explorer — light theme, phone portrait. Tiger card expanded showing full description. Heart icons visible on each card.

Figure 2: Android Studio showing AnimalFact data class (left) alongside the running Animal Explorer app in the emulator preview (right). Build: 33 tasks, 7 executed.

```kotlin
package com.example.animalfacts.data

15 Usages
data class AnimalFact(
    val id: Int,
    val name: String,
    val habitat: String,
    val imageResId: Int,
    val description: String,
    val isFavourite: Boolean = false
)
```

Figure 3: AnimalFact data class in Android Studio — package com.example.animalfacts.data, 6 fields, 15 usages tracked by the IDE.

4.6 Presentation Slide Content

Slide 1 — Application Overview

Title: Animal Facts Explorer — Kotlin & Jetpack Compose

- A scrollable educational app showcasing 10 animals with name, habitat, and facts
- Built with Kotlin, Jetpack Compose, Material 3, and MVVM architecture
- Key features: LazyColumn list, favourite toggling, expandable cards, light/dark theme
- Data class: AnimalFact (id, name, habitat, imageResId, description, isFavourite)
- State management: AnimalExplorerViewModel with MutableStateFlow<List<AnimalFact>>

Slide 2 — Theming Design & UI Screenshots

Title: Material 3 Brand Theme — Wildlife Identity

- Primary: Forest Green (#2E7D32 light / #81C784 dark) — brand identity and AppBar
- Tertiary: Amber (#F57F17 light / #FFB74D dark) — favourite heart icon accent
- Full theme architecture: Color.kt, Typography.kt, Shapes.kt, Theme.kt
- Dynamic color support on Android 12+; manual brand palette on older devices
- All composables reference MaterialTheme tokens — zero hardcoded colors
- [Insert light theme and dark theme screenshots side by side]

Slide 3 — Animation Flow & Key Learnings

Title: Purposeful Animation — Communicate, Don't Decorate

- Card expansion: animateContentSize() + AnimatedVisibility (fadeIn + expandVertically)
- Favourite toggle: animateColorAsState 300ms tween, tertiary color from theme
- Key learning 1: Animation targets must read from theme tokens to work correctly in both light and dark modes
- Key learning 2: Favourite state must live in ViewModel — local remember resets on scroll
- Key learning 3: key = { it.id } in LazyColumn items() is essential for correct recomposition tracking
- Future: search/filter bar, Paging 3, swipe-to-favourite gesture, detail screen navigation

4.7 Reflection & Future Enhancements

Building Animal Explorer as a complete themed and animated Jetpack Compose app consolidated several concepts that were previously understood in isolation. The most important lesson was that theming and animation are not independent systems — they intersect every time an animated value references a theme color, and getting that intersection right (using MaterialTheme.colorScheme.tertiary as the animation target rather than a hardcoded hex value) required understanding both systems simultaneously. The MVVM architecture decision — to keep isFavourite in the ViewModel's StateFlow rather than in composable remember — was initially counter-intuitive because it required more boilerplate than a local state variable, but its correctness became obvious the first time a card was scrolled off-screen and scrolled back: the favourite state was preserved exactly as expected, while a local remember implementation would have silently reset it. The key = { it.id } parameter in LazyColumn items was another lesson learned through observation: the Compose layout inspector clearly showed unnecessary recompositions when the key was absent and a favourited item changed position, and the fix was a single parameter addition. Future enhancements planned for Animal Explorer include: a search TextField at the top of the list filtering animals by name or habitat using derivedStateOf; a Favourites filter chip showing only isFavourite == true animals; a detail screen navigated via NavController with a shared element transition on the animal image; infinite scroll using Paging 3 loading from a REST wildlife API; and offline caching using Room so favourites and animal data persist across app restarts. Each of these enhancements builds directly on the architecture established in this session, demonstrating that investing in correct foundations — clean data models, ViewModel state ownership, semantic theme tokens — makes every future feature faster and safer to implement.